

Memory

Index

- Memory Hierarchy
- Cache: Basics, Mapping
- Cache: Performance, Associativity
- Multilevel Caches
- Virtual Memory: Basics, Page Tables & TLBs
- Integrating Virtual Memory, TLBs, Caches

Memory Technology

- Static RAM (SRAM)
0.5ns – 2.5ns, \$2000 – \$5000 per GB
- Dynamic RAM (DRAM)
50ns – 70ns, \$20 – \$75 per GB
- Magnetic disk
5ms – 20ms, \$0.20 – \$2 per GB
- Ideal memory
 - Access time of SRAM
 - Capacity and cost/GB of disk

Principle of Locality

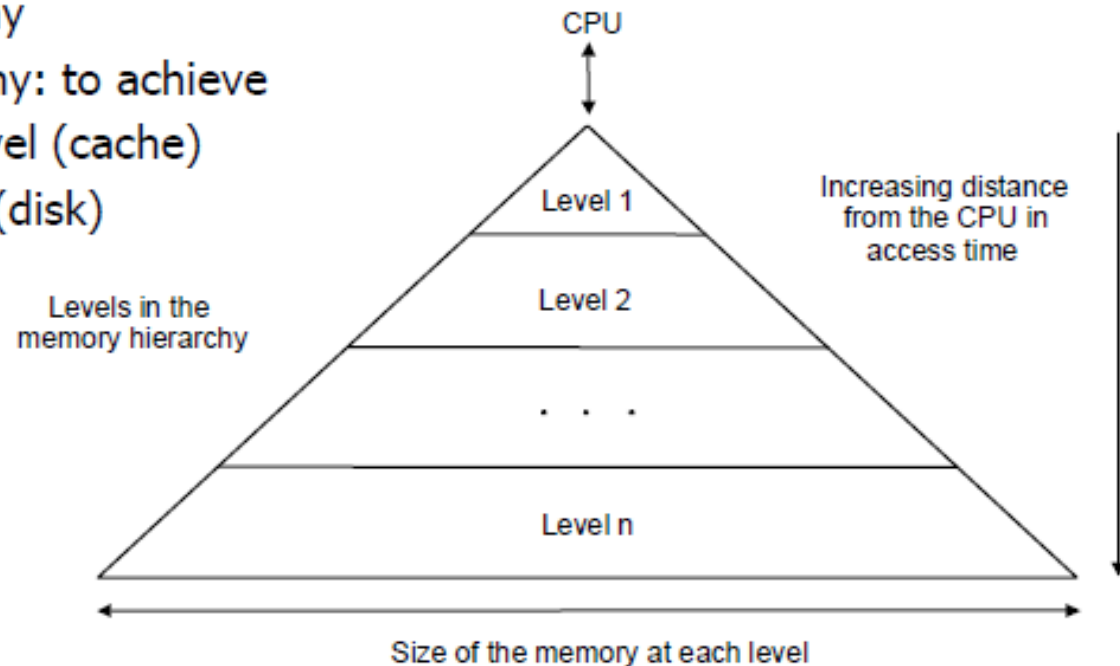
- Programs access a small proportion of their address space at any time
- **Temporal locality**
 - Items accessed recently are likely to be accessed again soon
 - e.g., instructions in a loop
- **Spatial locality**
 - Items near those accessed recently are likely to be accessed soon
 - E.g., sequential instruction access, array data

Taking Advantage of Locality

- Memory hierarchy
- Store everything on disk
- Copy recently accessed (and nearby) items from disk to smaller DRAM memory ==> Main memory
- Copy more recently accessed (and nearby) items from DRAM to smaller SRAM memory ==> Cache memory attached to CPU

Building a Memory Hierarchy

- Build a memory hierarchy
- Goal of memory hierarchy: to achieve
 - speed of highest level (cache)
 - size of lowest level (disk)

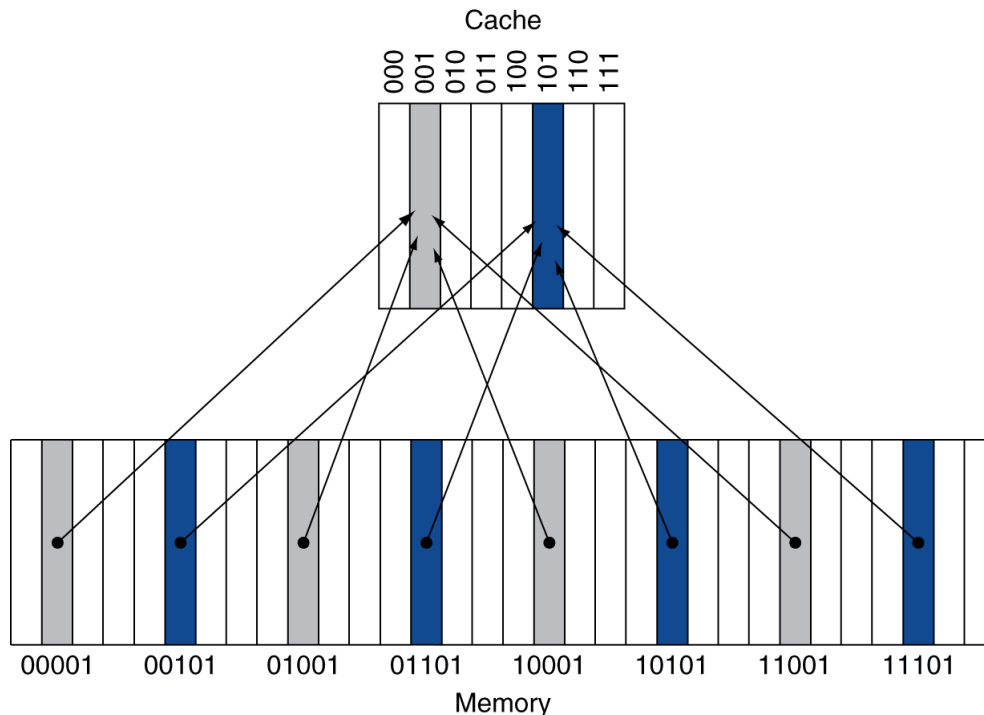


Cache Memory

- Cache memory
 - The level of the memory hierarchy closest to the CPU
- Given accesses $X_1, \dots, X_{n-1}, X_n$
 - How do we know if the data is present?
 - Where do we look?
 - 1. Direct Mapped
 - 2. Set Associative
 - 3. Fully Associative

Direct Mapped Cache

- Location determined by address
- Block size is one word of data
- Direct mapped: **only one choice**
(Block address) modulo (#Blocks in cache)



- #Blocks is a power of 2
- Use low-order address bits

Tags and Valid Bits

- How do we know which particular block is stored in a cache location?
 - Store block address as well as the data
 - Actually, only need the high-order bits
 - Called the tag
- What if there is no data in a location?
 - Valid bit: 1 = present, 0 = not present
 - Initially 0

Cache Example

- 8-blocks, 1 word/block, direct mapped
- Initial state

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	N		
111	N		

Cache Example

Word addr	Binary addr	Hit/miss	Cache block
22	10 110	Miss	110

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Cache Example

Word addr	Binary addr	Hit/miss	Cache block
26	11 010	Miss	010

Index	V	Tag	Data
000	N		
001	N		
010	Y	11	Mem[11010]
011	N		
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Cache Example

Word addr	Binary addr	Hit/miss	Cache block
22	10 110	Hit	110
26	11 010	Hit	010

Index	V	Tag	Data
000	N		
001	N		
010	Y	11	Mem[11010]
011	N		
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Cache Example

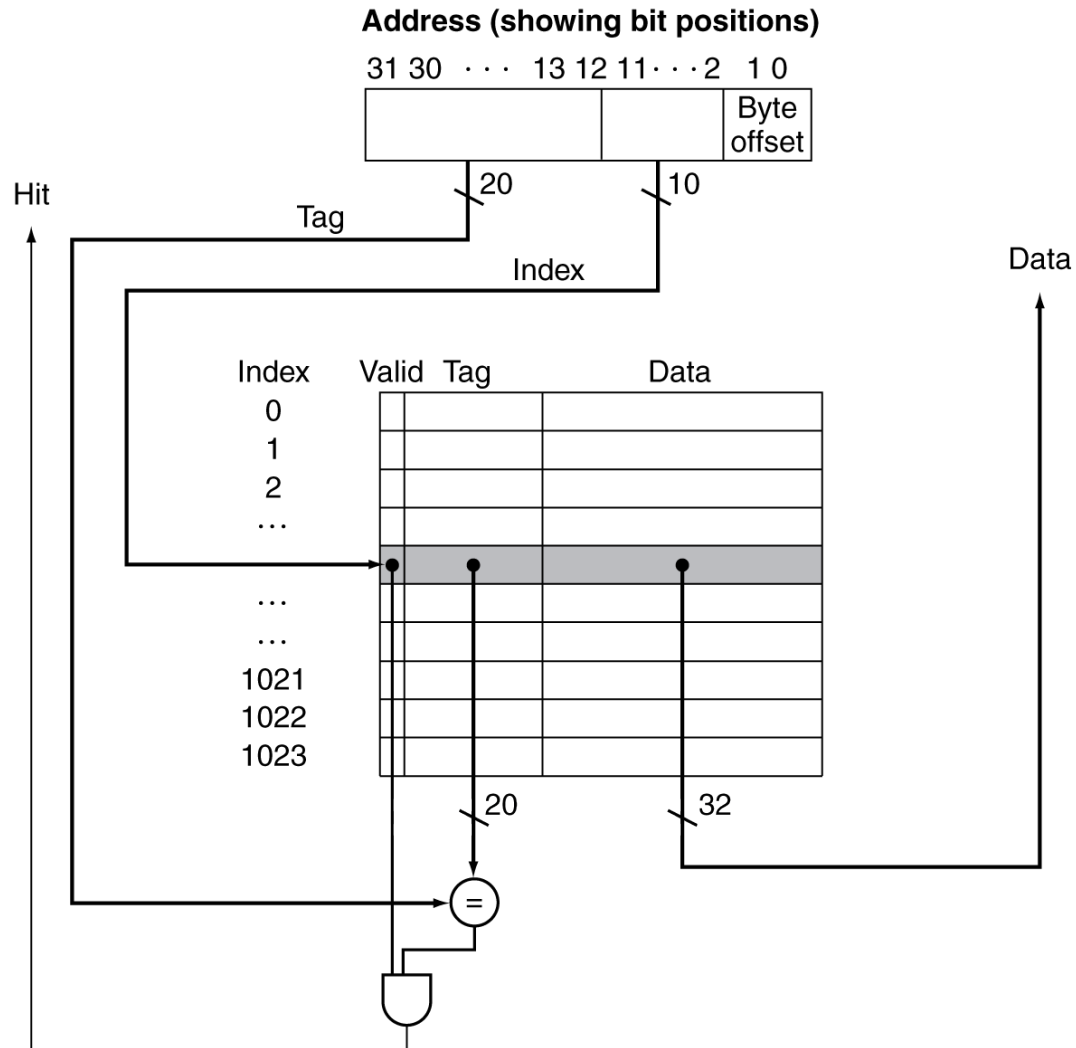
Word addr		Binary addr	Hit/miss	Cache block
16		10 000	Miss	000
3		00 011	Miss	011
16		10 000	Hit	000
Index	V	Tag	Data	
000	Y	10	Mem[10000]	
001	N			
010	Y	11	Mem[11010]	
011	Y	00	Mem[00011]	
100	N			
101	N			
110	Y	10	Mem[10110]	
111	N			

Cache Example

Word addr	Binary addr	Hit/miss	Cache block
18	10 010	Miss	010

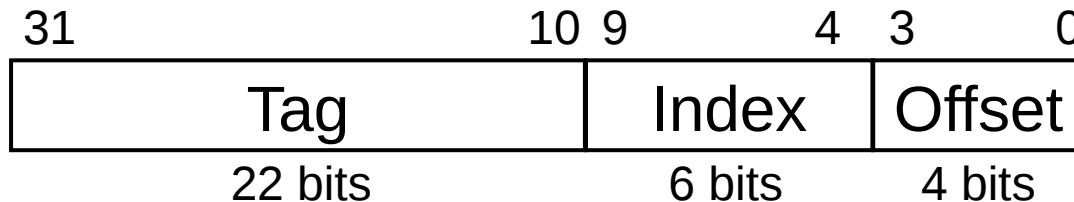
Index	V	Tag	Data
000	Y	10	Mem[10000]
001	N		
010	Y	10	Mem[10010]
011	Y	00	Mem[00011]
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Address Subdivision



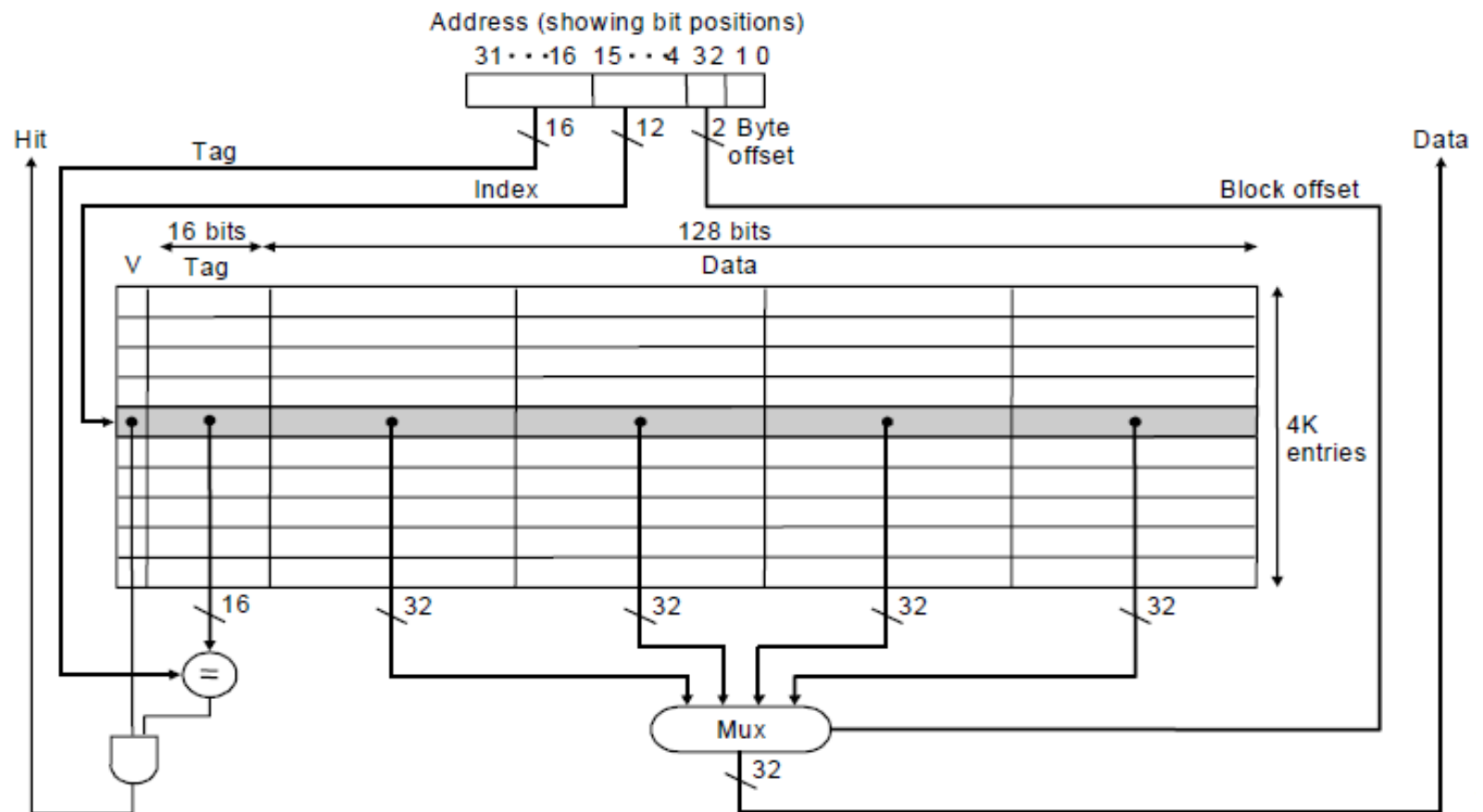
Example: Larger Block Size

- 64 blocks, 16 bytes/block
 - To what block number does address 1200 map?
- Block address (word number) = $\lceil 1200/16 \rceil = 75$
- Block number = $75 \text{ modulo } 64 = 11$



Direct Mapped Cache: Addressing

- Accessing a word in a 64KB cache, with 4 words per block



Example 3

Memory = 32 blocks = 64 words, cache = 4 blocks = 8 words, block = 2 words

Hence 6-bit word address: bit5-bit3 = tag, bit2-bit1 = index, bit0 = block offset

Word access sequence with word addresses (here we ignore byte addresses): 101100, 110100, 101100, 101101, 100001, 101100 (or memory word 44, 52, 44, 45, 33, 44)

Access sequence (block-word): 22-0, 26-0, 22-0, 22-1, 16-1, 22-0

Note: For each miss, 1 block (2 words) is transferred from memory to cache

index	V	tag	Word0	Word1
00	N			
01	N			
10	Y	101	Mem[44]	Mem[45]
11	N			

1st access: miss, transfer block 22

index	V	tag	Word0	Word1
00	N			
01	N			
10	Y	101	Mem[44]	Mem[45]
11	N			

3rd access: miss, transfer block 22, replace block 26

index	V	tag	Word0	Word1
00	Y	100	Mem[32]	Mem[33]
01	N			
10	Y	101	Mem[44]	Mem[45]
11	N			

5th access: miss, transfer block 16

index	V	tag	Word0	Word1
00	N			
01	N			
10	Y	110	Mem[52]	Mem[53]
11	N			

2nd access: miss, transfer block 26, replace block 22

index	V	tag	Word0	Word1
00	N			
01	N			
10	Y	101	Mem[44]	Mem[45]
11	N			

4th access: hit

index	V	tag	Word0	Word1
00	Y	100	Mem[32]	Mem[33]
01	N			
10	Y	101	Mem[44]	Mem[45]
11	N			

6th access: hit

Block Size Considerations

- Larger blocks should reduce miss rate
 - Due to spatial locality
- But in a fixed-sized cache
 - Larger blocks $\Rightarrow$ fewer of them
 - More competition $\Rightarrow$ increased miss rate
- Larger miss penalty
 - Can override benefit of reduced miss rate
 - Early restart and critical-word-first can help. Don't wait for full block to be loaded before restarting the CPU.

Cache Misses

- On cache hit, CPU proceeds normally
- On cache miss
 - Stall the CPU pipeline
 - Fetch block from next level of hierarchy
 - Instruction cache miss
 - Restart instruction fetch
 - Data cache miss
 - Complete data access

Write-Hit: Write-Through

On **data-write hit**, could just update the block in cache

- But then cache and memory would be inconsistent
- Sol: **Write through, Write Back**
- Write through: also update memory
- But makes writes take longer
 - e.g., if base CPI = 1, 10% of instructions are stores, write to memory takes 100 cycles
 - Effective CPI = $1 + 0.1 \times 100 = 11$
- Solution: write buffer
 - Holds data waiting to be written to memory
 - CPU continues immediately
 - Only stalls on write if write buffer is already full

Write-Hit: Write-Back

- Alternative: On data-write hit, just update the block in cache
 - Keep track of whether each block is dirty
- When a dirty block is replaced
 - Write it back to memory

Write-Miss: Write Allocation

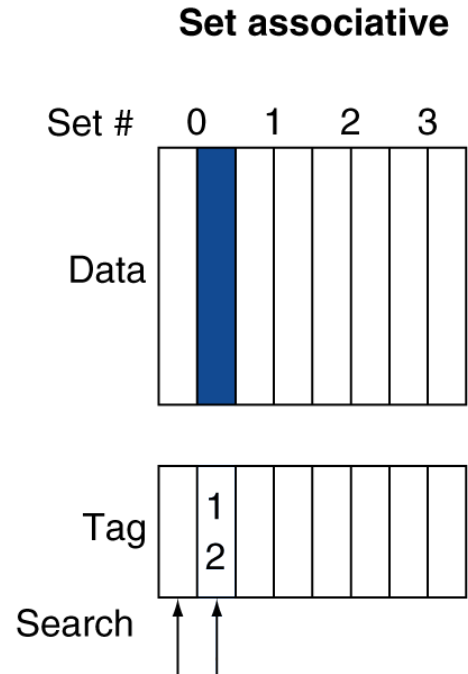
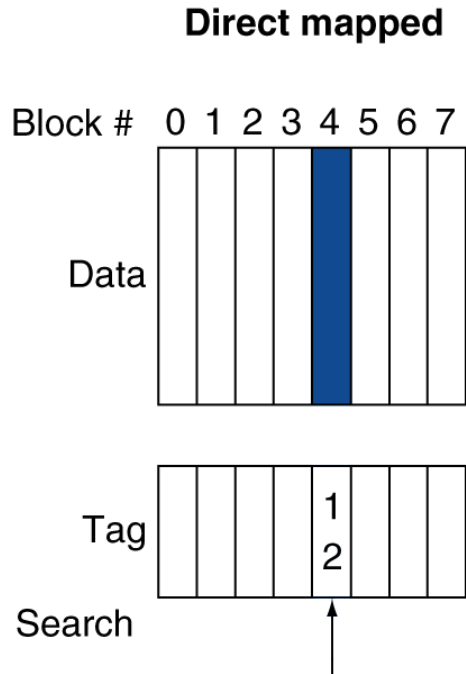
- What should happen on a **write miss**?
 - **Allocate on miss**: fetch the block
 - Block is loaded to the cache and then written into
 - **Write around**: don't fetch the block
 - Since programs often write a whole block before reading it (e.g., initialization)
 - The block is modified in L2/MM and not loaded into the cache

Associative Caches

Advantages/Disadvantages

- Fully associative
 - Allow a given block to go in any cache entry
 - Requires all entries to be searched at once
 - Comparator per entry (expensive)
- n -way set associative
 - Each set contains n entries
 - Block number determines which set
 - (Block number) modulo (#Sets in cache)
 - Search all entries in a given set at once
 - n comparators (less expensive)

Associative Cache Example



Spectrum of Associativity

- For a cache with 8 entries

**One-way set associative
(direct mapped)**

Block	Tag	Data
0		
1		
2		
3		
4		
5		
6		
7		

Two-way set associative

Set	Tag	Data	Tag	Data
0				
1				
2				
3				

Four-way set associative

Set	Tag	Data	Tag	Data	Tag	Data	Tag	Data
0								
1								

Eight-way set associative (fully associative)

Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data

Associativity Example

- Compare 4-block caches
 - Direct mapped, 2-way set associative, fully associative
 - Block access sequence: 0, 8, 0, 6, 8
- Direct mapped

Block address	Cache index	Hit/miss	Cache content after access			
			0	1	2	3
0	0	miss	Mem[0]			
8	0	miss	Mem[8]			
0	0	miss	Mem[0]			
6	2	miss	Mem[0]		Mem[6]	
8	0	miss	Mem[8]		Mem[6]	

Associativity Example

- 2-way set associative

Block address	Cache index	Hit/miss	Cache content after access		
			block 0		block 1
0	0	miss	Mem[0]		
8	0	miss	Mem[0]	Mem[8]	
0	0	hit	Mem[0]	Mem[8]	
6	0	miss	Mem[0]	Mem[6]	
8	0	miss	Mem[8]	Mem[6]	

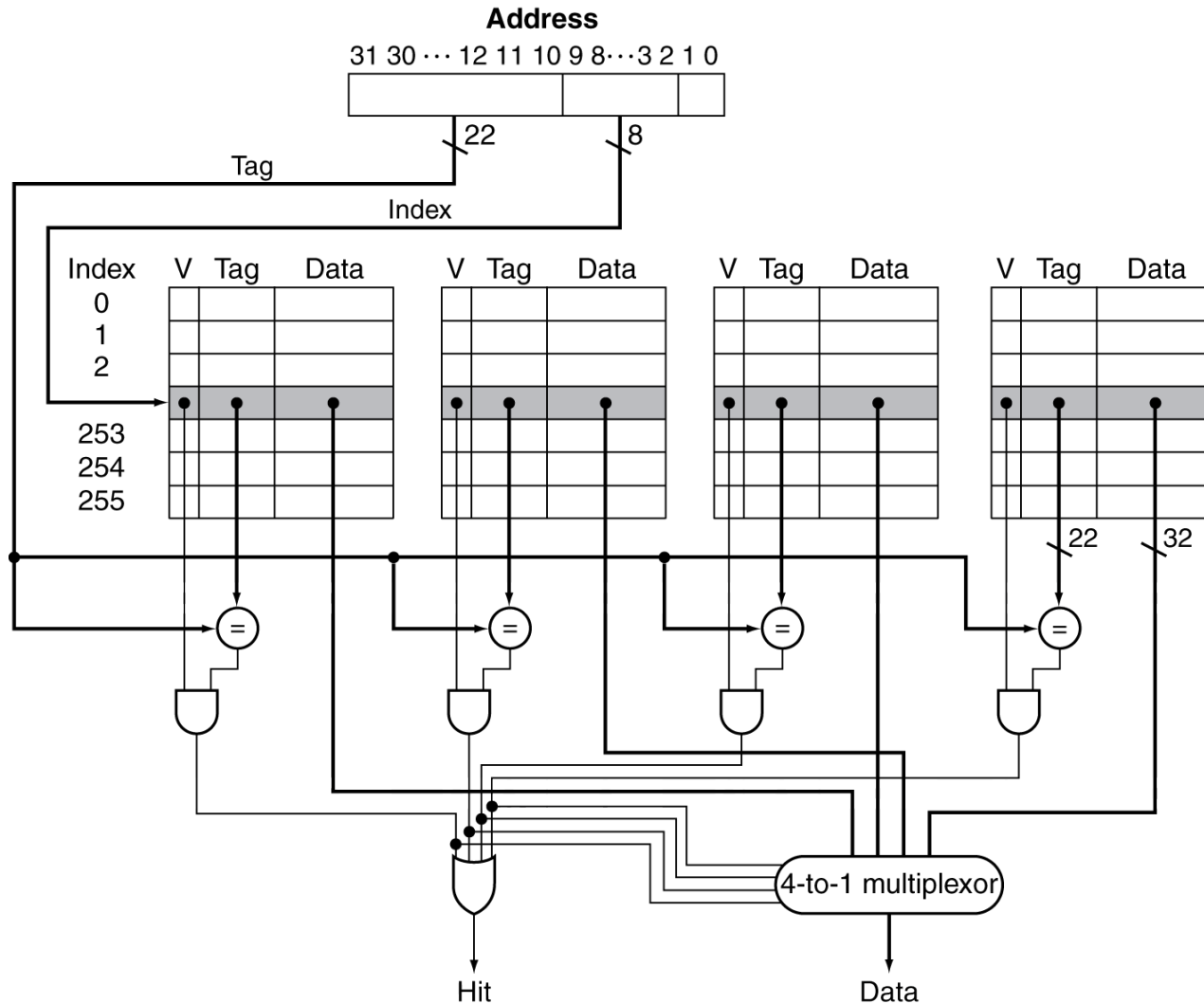
- Fully associative

Block address		Hit/miss	Cache content after access		
0		miss	Mem[0]		
8		miss	Mem[0]	Mem[8]	
0		hit	Mem[0]	Mem[8]	
6		miss	Mem[0]	Mem[8]	Mem[6]
8		hit	Mem[0]	Mem[8]	Mem[6]

How Much Associativity

- Increased associativity decreases miss rate
 - But with diminishing returns
- Simulation of a system with 64KB D-cache, 16-word blocks, SPEC2000
 - 1-way: 10.3%
 - 2-way: 8.6%
 - 4-way: 8.3%
 - 8-way: 8.1%

Set Associative Cache Organization



Replacement Policy

- Direct mapped: no choice
- Set associative
 - Least-recently used (LRU)
 - Choose the one unused for the longest time
 - Simple for 2-way, manageable for 4-way, too hard beyond that
 - Random
 - Gives approximately the same performance as LRU for high associativity

Multilevel Caches

- Primary cache attached to CPU
 - Small, but fast
- Level-2 cache services misses from primary cache
 - Larger, slower, but still faster than main memory
- Main memory services L-2 cache misses
- Some high-end systems include L-3 cache

Multilevel Cache Example

- Given
 - CPU base CPI = 1, clock rate = 4GHz
 - Miss rate/instruction = 2%
 - Main memory access time = 100ns
- With just primary cache
 - Miss penalty = $100\text{ns} / 0.25\text{ns} = 400$ cycles
 - Effective CPI = $1 + 0.02 \times 400 = 9$

Example (cont.)

- Now add L-2 cache
 - Access time = 5ns
 - Global miss rate to main memory = 0.5%
- Primary miss with L-2 hit
 - Penalty = $5\text{ns} / 0.25\text{ns} = 20$ cycles
- Primary miss with L-2 miss
 - Extra penalty = 400 cycles
- $\text{CPI} = 1 + 0.02 \times 20 + 0.005 \times 400 = 3.4$
- Performance ratio = $9 / 3.4 = 2.6$

Multilevel Cache Considerations

- Primary cache
 - Focus on minimal hit time
- L-2 cache
 - Focus on low miss rate to avoid main memory access
 - Hit time has less overall impact

Virtual Memory

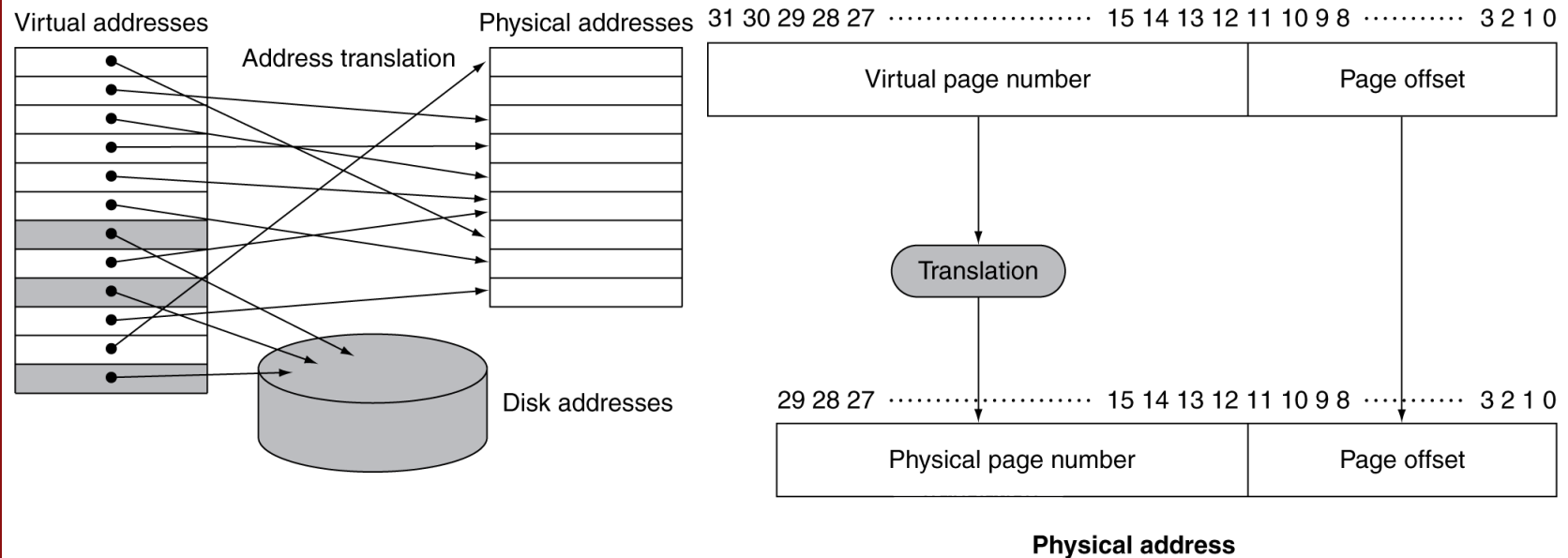
- Use main memory as a “cache” for secondary (disk) storage
 - Managed jointly by CPU hardware and the operating system (OS)
- Programs share main memory
 - Each gets a private virtual address space holding its frequently used code and data
 - Protected from other programs
- Conceptually same as a cache
- CPU and OS translate virtual addresses to physical addresses
 - VM “block” is called a page
 - VM translation “miss” is called a page fault

Virtual Memory

- Advantages:
 - illusion of having more physical memory
 - program relocation (maps virtual addresses from program to physical addresses in memory)
 - Protection (VM manages main memory/disk interactions)
 - Allows sharing of data

Address Translation

- Fixed-size pages (e.g., 4K) Virtual address



Address translation (memory mapping):
 CPU produces **virtual address**,
 translated by hardware / software to
physical address, which can be used to
 access main memory

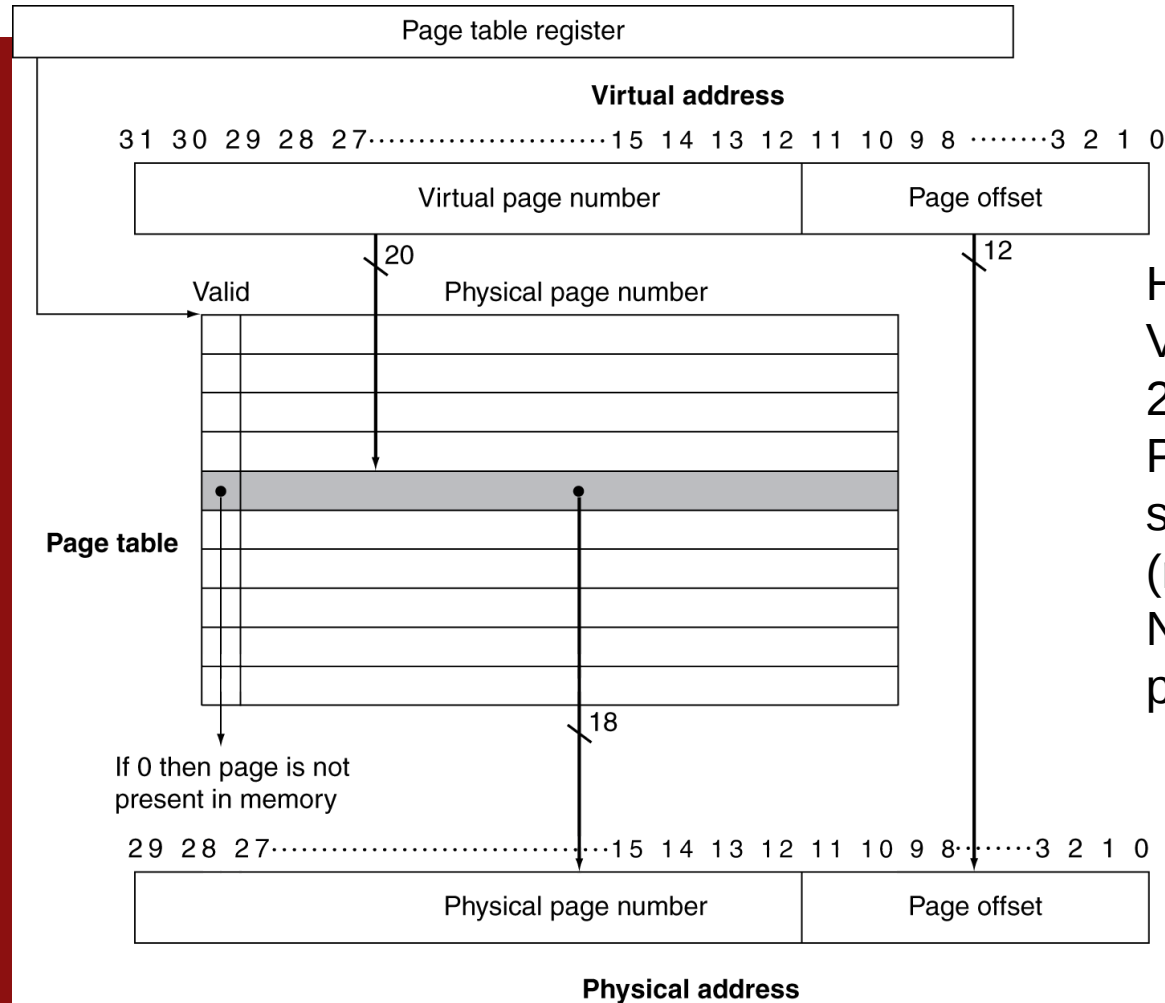
Page Fault Penalty

- On page fault, the page must be fetched from disk
 - Takes millions of clock cycles
 - Handled by OS code
- Try to minimize page fault rate
 - Fully associative placement (lower miss rate but higher hit cost)
 - Smart replacement algorithms (Index to memory: Page Tables)

Page Tables

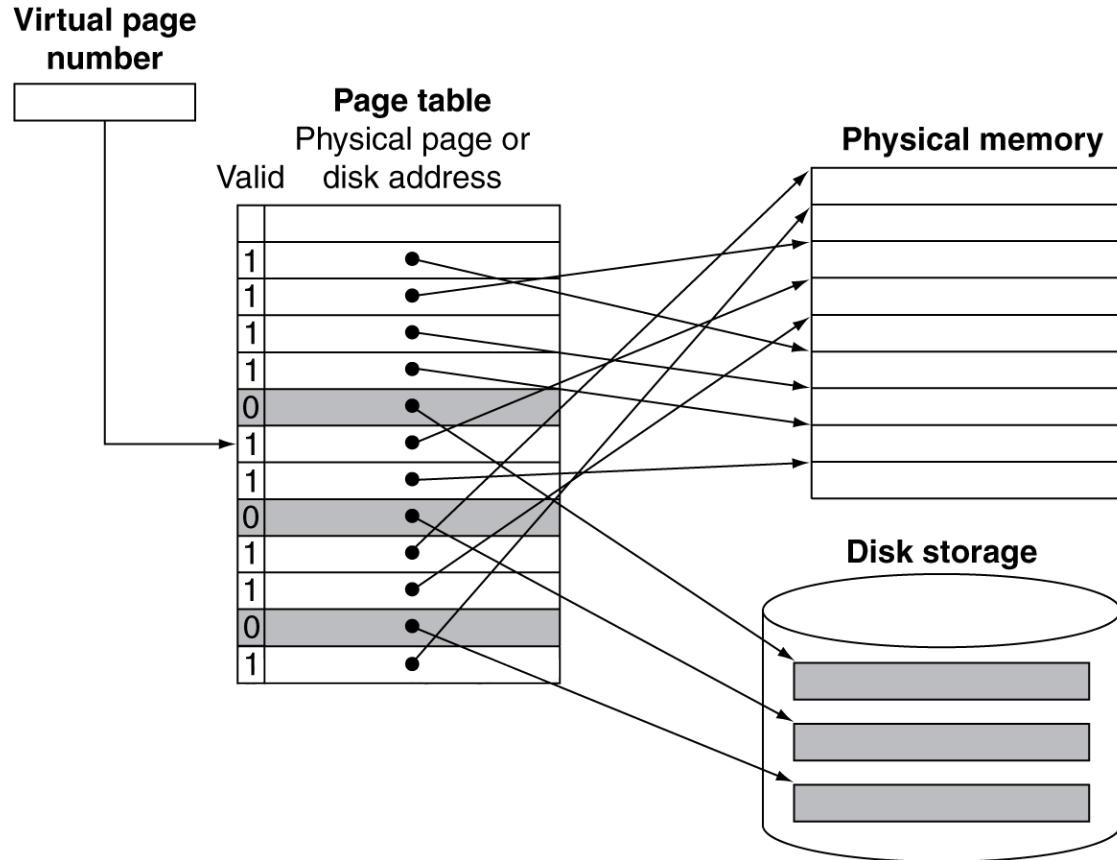
- Stores placement information
 - Page Tables reside in memory
 - Array of page table entries, indexed by virtual page number
 - Page table register in CPU points to page table in physical memory
- If page is present in memory
 - PTE stores the physical page number
 - Plus other status bits (referenced, dirty, ...)
- If page is not present
 - PTE can refer to location in swap space on disk

Translation Using a Page Table



Here:
Virtual address space:
 2^{32} bytes (4 GB)
Physical address
space: 2^{30} bytes
(memory up to 1 GB)
Number of entries in
page table: 2^{20}

Mapping Pages to Storage



Replacement and Writes

- To reduce page fault rate, prefer least-recently used (LRU) replacement
 - Reference bit (aka use bit) in PTE set to 1 on access to page
 - Periodically cleared to 0 by OS
 - A page with reference bit = 0 has not been used recently
- Disk writes take millions of cycles
 - Write through is impractical
 - Use write-back
 - Dirty bit in PTE set when page is written

Page Table

- Page fault: if valid bit in page table is off, a page fault occurs, and operating system gets control to find page in disk and decides where to place page in main memory (LRU or approximate LRU if all pages in memory are in use)

Example: 32-bit virtual address, 4 KB pages, 4 bytes per page table entry.
Page table size = ?

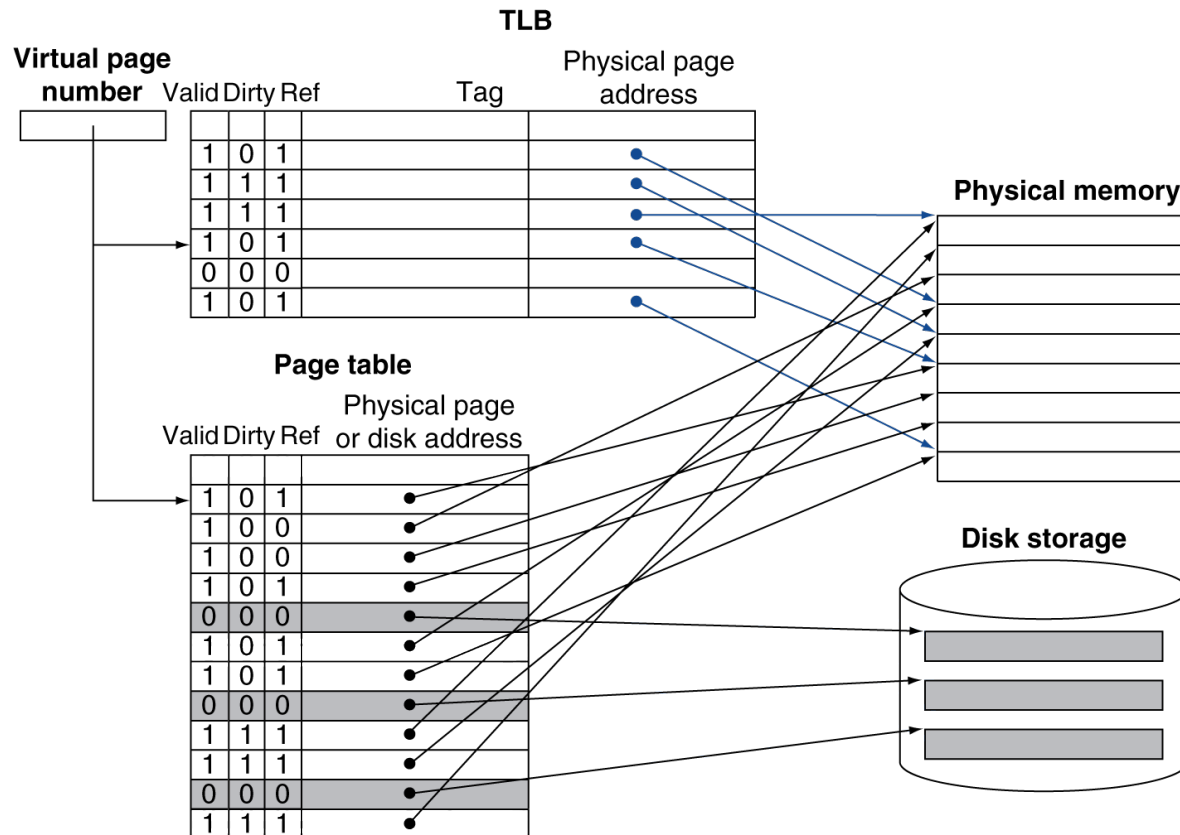
Solution: Page offset = 12 bits; number of page table entries = $2^{32} / 2^{12} = 2^{20}$

- Page table size = $2^{20} \times 4 = 4 \text{ MB}$
- Page table uses huge amount of memory --> techniques to limit page table size

Fast Translation Using a TLB

- Address translation would appear to require extra memory references
 - One to access the PTE
 - Then the actual memory access
- But access to page tables has good locality
 - So use a fast cache of PTEs within the CPU
 - Called a Translation Look-aside Buffer (TLB)
 - Typical: 16–512 PTEs, 0.5–1 cycle for hit, 10–100 cycles for miss, 0.01%–1% miss rate
 - Misses could be handled by hardware or software

Fast Translation Using a TLB



TLB is a cache (subset of page table) & therefore must have a tag field

On TLB miss, page table must then be examined

Typical TLB:

16 - 512 entries

each block of 1 - 2 entries

hit time of 0.5 - 1 cycle

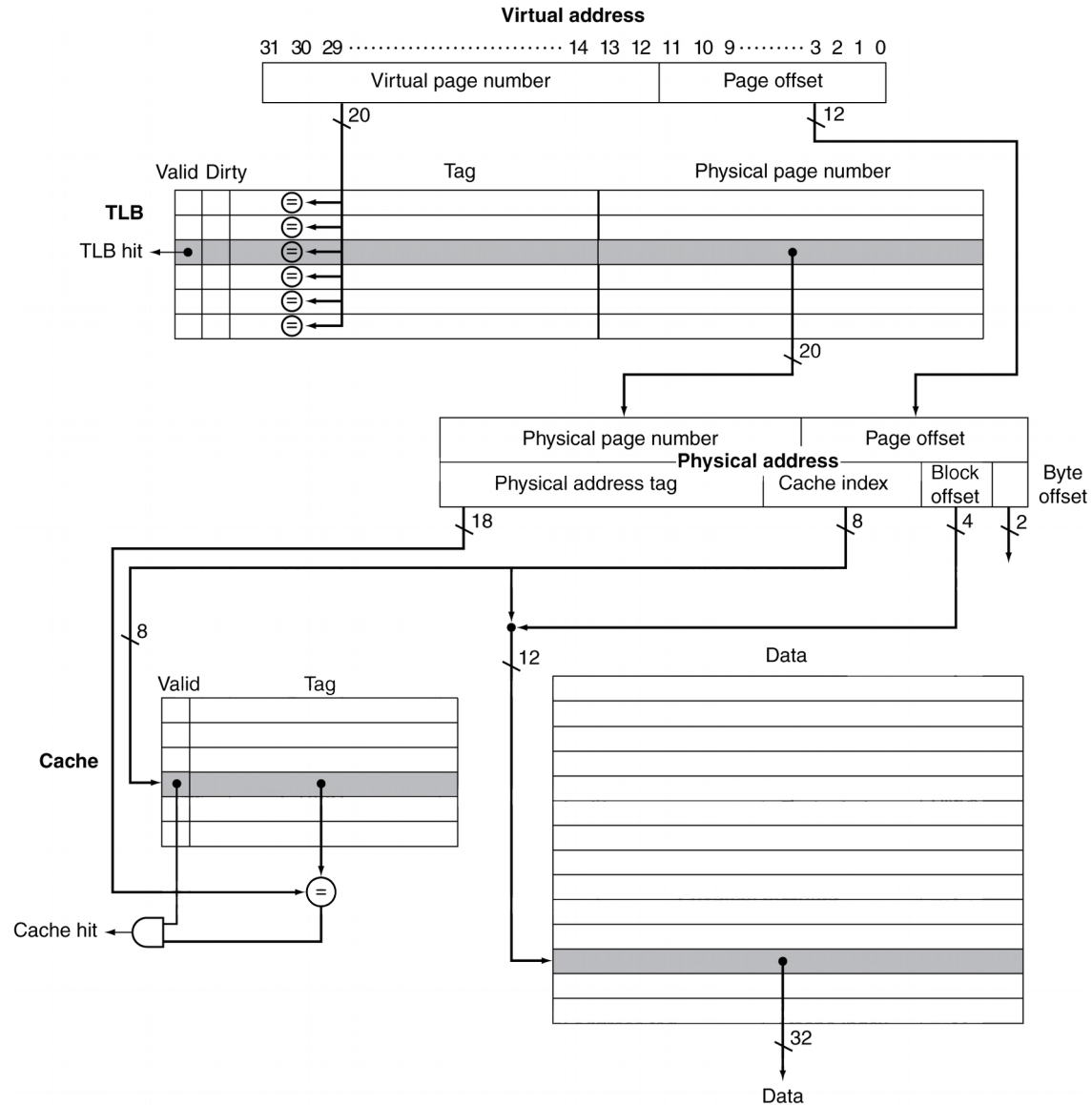
miss penalty of 10 - 100

cycles miss rate of 0.01 - 1%

TLB Misses

- If page is in memory
 - Load the PTE from memory and retry
 - Could be handled in hardware
 - Can get complex for more complicated page table structures
 - Or in software
 - Raise a special exception, with optimized handler
- If page is not in memory (page fault)
 - OS handles fetching the page and updating the page table
 - Then restart the faulting instruction

TLB and Cache Interaction



Virtual Memory example

TLB						
Index	Set 0			Set 1		
	Tag	PPN	Valid	Tag	PPN	Valid
0	03	C3	1	01	71	0
1	00	28	1	01	35	1
2	02	68	1	3A	F1	0
3	03	12	1	02	30	1
4	7F	05	0	01	A1	0
5	00	53	1	03	4E	1
6	1B	34	0	00	1F	1
7	03	38	1	32	09	0

Page Table		
VPN	PPN	Valid
000	71	1
001	28	1
002	93	1
003	AB	0
004	D6	0
005	53	1
006	1F	1
007	80	1
008	02	0
009	35	1
00A	41	0
00B	86	1
00C	A1	1
00D	D5	1
00E	8E	0
00F	D4	0
010	60	0
011	57	0
012	68	1
013	30	1
014	0D	0
015	2B	0
016	9F	0
017	62	0
018	C3	1
019	04	0
01A	F1	1
01B	12	1
01C	30	0
01D	4E	1
01E	57	1
01F	38	1

Imagine a system with the following parameters:

- **Virtual addresses:** 20 bits
- **Physical addresses:** 18 bits
- **Page size:** 1 KB
- **TLB:** 2-way set associative, 16 total entries
- **Single-level page table**

The contents of the TLB and the first 32 entries of the page table are shown as follows. All numbers are in hexadecimal.

Virtual Memory example

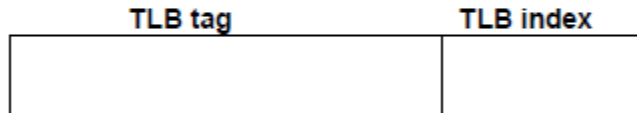
Part 1: Components of Virtual and Physical Addresses

1. **Virtual Address Fields.** The virtual address is 20 bits.

- a. Show the number of bits used for the **virtual page number** and the **virtual page offset** below.



- b. Show the number of bits used for the TLB tag and TLB index. **Label the bit numbers to denote the bit range.**



- c. Assuming a single-level paging system, how many entries are in the page table?

2. **Physical Address Fields.** The physical address is 18 bits.

- a. Show the number of bits used for the **physical page number** and the **physical page offset**. Label the bit numbers to denote the bit range.



Virtual Memory example

For the given virtual addresses, indicate:

1. Which TLB entry is accessed
2. The physical address which is formed
3. Whether the TLB access was a hit or a miss
4. Whether a page fault occurs
5. The physical page number

1. **Virtual address:** 0x078E6

a. **Virtual address in binary** (one bit per box)

19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

2. **Address translation**

Parameter	Value
Virtual Page Number	0x
TLB Index	0x
TLB Tag	0x
TLB Hit? (Hit/Miss)	
Page Fault? (yes/no)	
Physical page number	0x

3. **Physical address** (one bit per box)

17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0